

Detecting Dirty Dish Culprits with Computer Vision

Kevin Yin
University of Michigan
500 S State St, Ann Arbor
kevin Yin@umich.edu

Joshua Bisdorf
University of Michigan
500 S State St, Ann Arbor
bisdorfj@umich.edu

1. Introduction

We have developed a system that outputs the time stamps in a video when someone places down dirty dishes in our fraternity house’s sink or drying rack without cleaning them.

1.1. Motivation

The authors (Kevin Yin and Joshua Bisdorf) are members of a fraternity and have lived with their fellow members in a fraternity house. The fraternity house uses the Swann Security system, and there is a camera that faces the direction of the sink where the members are to clean and rack their dishes. Sometimes, the fraternity members fail to clean their dishes and drop dirty dishes off into the sink or drying rack. When enough dirty dishes pile up, an officer in the fraternity threatens to check the camera footage and uncover who left behind dirty dishes. However, the officers rarely follow through with this because it is incredibly tediousness to sift through days worth of footage. Our goal is to turn this into a simple two step process. First, to download the video data from the security system. And second, to run the footage through our programs and output the time stamps of when people put back dirty dishes without cleaning them.

1.2. Scope of the Problem

This problem is very specific to the footage of our fraternity house’s kitchen. This is not because our model can not be applied to other kitchen sinks for example. It is because all of our training data comes from our kitchen, as that is the only large enough source of data we have access to train with. In theory, our methods can be applied elsewhere, but that is beyond the scope of our original motivation.

1.3. Contributions

This problem is shared by the countless people in the world who share kitchens with roommates. There is similar work that will be discussed in the section Related Work. However, ours is unique in that it focuses on the person in

the video doing the dishes, not the dishes themselves. This is because the camera in the kitchen is angled to view the backs of the people standing at the sink. For the most part, their bodies block the dishes. We wanted to make this model that fits towards our data specifically because it was the only data that was relevant to our goal.

2. Related Work

2.1. Dishcraft Robotics [1]

Dishcraft Robotics has introduced a system aimed at automating dishwashing in commercial kitchens. This system utilizes computer vision to inspect each dish for residual food particles, ensuring consistent cleanliness. Dishes that do not meet the cleanliness criteria are reprocessed. However, the system requires the use of custom dishes with magnetic bases, which may limit its applicability in diverse kitchen environments.

2.2. Detecting Dirty Dishes with OpenCV [3]

Developed by a member of the London Hackspace, this system employs a camera mounted above the sink and utilizes the HoughCircles function from the OpenCV library to detect circular items such as plates and glasses. When dishes remain in the sink beyond a predetermined time, it sends alerts via a traffic light system or IRC notifications. A limitation of this approach is its reliance on detecting circular shapes, which may not account for non-circular dishes or cutlery.

2.3. DISH-O-TRON [2]

The DISH-O-TRON system uses a computer vision AI model to detect dirty dishes in communal kitchen sinks. Upon detection, it emits a beeping sound to prompt users to clean up. The project emphasizes data collection, model training, and deployment on edge devices to respect user privacy. However, the system’s effectiveness can be influenced by varying lighting conditions and the diversity of dish types.

	from	n	params	module	arguments
0	-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]
1	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
2	-1	1	29856	ultralytics.nn.modules.block.C2f	[64, 64, 1, True]
3	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
4	-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
5	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
6	-1	2	788480	ultralytics.nn.modules.block.C2f	[256, 256, 2, True]
7	-1	1	1188672	ultralytics.nn.modules.conv.Conv	[256, 512, 3, 2]
8	-1	1	1838880	ultralytics.nn.modules.block.C2f	[512, 512, 1, True]
9	-1	1	656896	ultralytics.nn.modules.block.SPPF	[512, 512, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12	-1	1	591360	ultralytics.nn.modules.block.C2f	[768, 256, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15	-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
16	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18	-1	1	403856	ultralytics.nn.modules.block.C2f	[384, 256, 1]
19	-1	1	590336	ultralytics.nn.modules.conv.Conv	[256, 256, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21	-1	1	1969152	ultralytics.nn.modules.block.C2f	[768, 512, 1]
22	[15, 18, 21]	1	2116822	ultralytics.nn.modules.head.Detect	[2, [128, 256, 512]]

Model summary: 225 layers, 11,136,374 parameters, 11,136,358 gradients, 28.6 GFLOPs

Figure 1. Model

2.4. Comparison to Current Approaches

The other work focus on detecting the presence of dirty dishes using computer vision techniques. They often rely on specific object shapes or require custom dishware. In contrast, our approach aims to identify the individuals responsible for leaving dirty dishes by analyzing video footage to detect when someone places unclean dishes in the sink or drying rack. This person-centric method addresses the accountability aspect, which is not the primary focus of existing systems. This also means that the fraternity house's security camera does not need to be readjusted to focus on the dishes; it can remain in its current position. A con of this implementation is that it cannot be easily applied to different kitchens without the need for new training data.

3. Method

Our method for creating this project can be split into 3 main categories: data processing, training, and testing. To begin, we acquired 22 GB of MP4 video data as well as produced additional samples to ensure balancing of our labels. We then annotated relevant sections of this footage with bounding boxes to indicate two important objects: washers and walkers, with a washer being defined as a person doing the task of washing their dishes whereas a walker is defined as someone in frame who is not washing their dishes. We annotated this data using the Roboflow [4] annotation software and trained this model using transfer learning, with the original object detection model being the publicly available yolov8-s model, as shown in figure 1. This is a basic object detection model that allows us to transfer basic object representations and then fit them to our task of detecting people washing their dishes.

We trained the model for 20 epochs with an Adam optimizer with a learning rate of 0.001667 and a momentum of 0.9 as well as a Cross Entropy Loss function, with the graphs shown in Figure 2. below. Through this, our model is able to detect frames in which people are washing their dishes. The next step of this model is to process this output

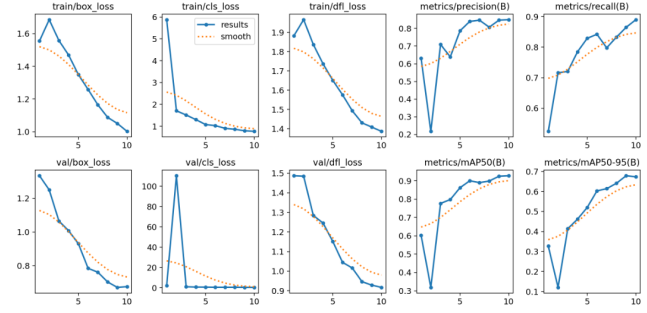


Figure 2. Training Performance Measurements Graphs

data to determine when people are just placing their dishes into the sink without washing them. To achieve this, our model tracks the number of consecutive frames that a person is detected as "washing", with minor room to account for noise. If a person is detected to being washing for too short of a time, then the algorithm will print the frame where they began washing for further human review. The decision to split our model into two parts was also very important, an idea inspired by implementations of action recognition found online that split the model first into a CNN to learn object detection followed by an RNN for learning from temporal representations.

4. Experiments

Firstly, our algorithm included a train, validation, and test split in a ratio of 70/20/10. This ensured our model did not over fit or under fit to any particular data points. Every run, we measured loss using a cross binary loss function and worked to minimize this. Furthermore, we recorded new data where we meticulously acted out faking and washing dishes to ensure that our model worked correctly. We split this video data into segments to determine if individual detections for washing or washing for too low of a time were detected correctly in multiple situations. We then tested our model on the whole of the data to check if it worked robustly for our action classification task as a way to manually check the performance of our model until satisfactory results were achieved, with a final mAP50 score of .96.

5. Conclusion

In the end, our model was able to detect if people were washing their dishes with very high fidelity. We found that by transfer learning, our model was able to learn feature representations of objects with a much greater accuracy than if we were to have trained our model from scratch. That said, we were afforded a great amount of data to learn that was importantly all very similar, simplifying the process of our model's learning by a great margin and allowing for better results to achieved with less training.

References

- [1] Evan Ackerman. Dishcraft robotics takes over dishwashing from humans, 2019. IEEE Spectrum, June 18, 2019. Available at <https://spectrum.ieee.org/dishcraft-robotics-takes-over-dishwashing-from-humans>.
- [2] Marcel Mikl and Oliver Moser. Dish-o-tron – no more dirty dishes thanks to ai, 2020. codecentric AG, September 10, 2020. Available at <https://www.codecentric.de/wissens-hub/blog/dish-o-tron-no-more-dirty-dishes-ai>.
- [3] Matt Richardson. Detecting dirty dishes with opencv, 2012. Make: Magazine, January 20, 2012. Available at <https://makezine.com/article/craft/detecting-dirty-dishes-with-opencv>.
- [4] Roboflow. Roboflow: Computer vision tools for developers and enterprises, 2024. Available at <https://roboflow.com/>.